

Support Material for “CGBDSM: An Efficient Batch-Dynamic Subgraph Matching System on CPU-GPU Platforms”

This support material gives the complete pseudocode for the general edge-triggered BDSM framework, asynchronous batch pipeline, batch insertion, activation, deletion, validation, GPU staging, and staticized matching. It uses the notation introduced in the main paper.

Algorithm S1 formalizes the group-interleaved execution: deletion matching precedes retraction, insertion matching follows the update, and triggers execute in parallel only within the active query-edge group.

Algorithm S1 states the baseline edge-triggered semantics against which CGBDSM is contrasted. Line 1 builds the initial candidate index; lines 2–3 take and split one batch; and lines 4–6 form the label-compatible trigger groups for each anchor query edge. Lines 7–9 enumerate deletions before retraction, apply both structural changes, and then enumerate insertions. Lines 10–12 launch one independent search per trigger in the current group. Lines 13–17 are the depth-first enumerator: they report a complete mapping, intersect candidate constraints, enforce injectivity, and recurse.

Algorithm S2 separates CPU maintenance from GPU matching without changing batch order. Lines 1–6 build the two indices, FIFO, and result buffer, then spawn the producer and run the consumer. Lines 7–16 split every batch, maintain deletion and insertion in turn, export each compact task, enqueue it, and finally close the queue. Lines 17–23 dequeue tasks in FIFO order, dispatch the polarity-matched GPU routine, and emit batch i only after both results are present. Thus the producer advances the persistent BPDCI without waiting for matching, while FIFO gives every task its required predecessor projection.

BATCHINSERT converts an insertion sub-batch into new live candidate relations. Line 1 initializes the dependency delta and activation seeds; lines 2–7 inspect every inserted edge in both orientations, creating a dependency and seeding both endpoints when labels match. Lines 8–9 deduplicate the parallel output and install all structural relations. Lines 10–11 activate newly supported candidates and revalidate the remaining failure boundary. Lines 12–13 export the direct and newly exposed live relations and return the result.

BATCHACTIVATION is the forward, level-synchronous insertion fixpoint. Line 1 initializes the failure set, activation delta, and frontier level; lines 2–4 create a work set and inspect the current frontier in parallel. Lines 5–11 activate a supported inactive candidate, record it, and schedule its

Algorithm S1 General Edge-Triggered BDSM Execution

```

1:  $C \leftarrow \text{BuildIndex}(G^0, Q)$ 
2: for each batch  $\Delta B$  do
3:    $(\Delta B^+, \Delta B^-) \leftarrow \text{Split}(\Delta B)$ 
4:   for each anchor query edge  $e = (u_a, u_b) \in E(Q)$  do
5:      $\mathcal{T}^-(e) \leftarrow \{\Delta e \in \Delta B^- \mid (L(u_a), L(u_b)) = (L(v_a), L(v_b))\}$ 
6:      $\mathcal{T}^+(e) \leftarrow \{\Delta e \in \Delta B^+ \mid (L(u_a), L(u_b)) = (L(v_a), L(v_b))\}$ 
7:      $\Delta M^- \leftarrow \Delta M^- \cup \text{MATCH}(e, \mathcal{T}^-(e), C)$ 
8:      $C \leftarrow \text{Retract}(C, \mathcal{T}^-(e)); C \leftarrow \text{Update}(C, \mathcal{T}^+(e))$ 
9:      $\Delta M^+ \leftarrow \Delta M^+ \cup \text{MATCH}(e, \mathcal{T}^+(e), C)$ 
10: function MATCH( $e = (u_a, u_b), \mathcal{T}, C$ )
11:   for each  $\Delta e = (v_a, v_b) \in \mathcal{T}$  in parallel do
12:      $M \leftarrow \{u_a \mapsto v_a, u_b \mapsto v_b\}; \text{ENUM}(\varphi_e, C, M, 3)$ 
13:   function ENUM( $\varphi, C, M, j$ )
14:     if  $j = |\varphi| + 1$  then report  $M$ ; return
15:      $u \leftarrow \varphi[j]; W \leftarrow C_v(u) \cap \bigcap_{u' \in N_{\varphi}^-(u)} \{v \mid (M(u'), v) \in C_e(e(u', u))\}$ 
16:     for each  $v \in W \setminus M(V(Q))$  do
17:       ENUM( $\varphi, C, M \cup \{u \mapsto v\}, j + 1$ )

```

inactive dependency neighbors; lines 12–13 retain an unsupported candidate at the failure boundary. Lines 14–15 synchronize, deduplicate, and form the next frontier. Lines 16–17 remove stale failure entries and return the fixpoint result.

BATCHDELETE performs structural removal before propagating its semantic consequences. Line 1 initializes the removal delta and validation seeds. Lines 2–7 inspect every deleted edge in both directions, collect each label-compatible dependency, and seed its endpoint candidates. Lines 8–9 deduplicate the parallel collections and retain only currently active seeds. Lines 10–11 remove the records without reading states, then validate the affected candidates; lines 12–13 project direct and propagated retractions for matching.

BATCHVALIDATION is the reverse, level-synchronous fixpoint for candidates that lose active support. Line 1 initializes the retraction delta and supplied frontier; lines 2–4 allocate and process the next level in parallel. Lines 5–9 forward an already inactive candidate’s failure to its active dependency neighbors. Lines 10–16 mark an unsupported active candidate inactive, record it, and schedule those same neighbors. Lines 17–18 synchronize and deduplicate the next level; line 19 returns at convergence.

Algorithm S7 packages host-side relation changes into a GPU task and realizes the two boundary views. In EXPORT-TASK, lines 1–10 initialize compact partitions, extract each

Algorithm S2 Asynchronous CPU–GPU Batch Pipeline of CGBDSM

Input: initial graph G^0 , query Q , normalized batches $\Delta\mathcal{B}$

Output: $(\Delta\mathcal{M}_i^-, \Delta\mathcal{M}_i^+)$ for every batch $\Delta\mathcal{B}_i$

```

1: procedure ASYNCPipeline( $G^0, Q, \Delta\mathcal{B}$ )
2:    $C \leftarrow \text{BUILDINDEX}(G^0, Q)$ 
3:    $C_{\text{gpu}} \leftarrow \text{CONSTRUCTPROJECTION}(C)$ 
4:    $Q \leftarrow \text{CREATEFIFO}()$ ;  $R \leftarrow \emptyset$ 
5:    $\text{SPAWN}(\text{PRODUCE}, C, \Delta\mathcal{B}, Q)$ 
6:    $\text{CONSUME}(C_{\text{gpu}}, Q, R)$ 
7: procedure PRODUCE( $C, \Delta\mathcal{B}, Q$ )
8:   for  $i \leftarrow 1$  to  $|\Delta\mathcal{B}|$  do
9:      $(\Delta\mathcal{B}_i^-, \Delta\mathcal{B}_i^+) \leftarrow \text{SPLIT}(\Delta\mathcal{B}_i)$ 
10:     $(C, \Delta D_i^-) \leftarrow \text{BATCHDELETE}(C, \Delta\mathcal{B}_i^-)$ 
11:     $\tau_i^- \leftarrow \text{EXPORTTASK}(\Delta D_i^-, \Delta\mathcal{B}_i^-)$ 
12:     $\text{ENQUEUE}(Q, \langle i, -, \tau_i^- \rangle)$ 
13:     $(C, \Delta D_i^+) \leftarrow \text{BATCHINSERT}(C, \Delta\mathcal{B}_i^+)$ 
14:     $\tau_i^+ \leftarrow \text{EXPORTTASK}(\Delta D_i^+, \Delta\mathcal{B}_i^+)$ 
15:     $\text{ENQUEUE}(Q, \langle i, +, \tau_i^+ \rangle)$ 
16:   $\text{CLOSE}(Q)$ 
17: procedure CONSUME( $C_{\text{gpu}}, Q, R$ )
18:   while  $\langle i, s, \tau \rangle \leftarrow \text{DEQUEUE}(Q) \neq \perp$  do
19:     if  $s = -$  then
20:        $(C_{\text{gpu}}, R[i, -]) \leftarrow \text{MATCHDELETION}(C_{\text{gpu}}, \tau)$ 
21:     else
22:        $(C_{\text{gpu}}, R[i, +]) \leftarrow \text{MATCHINSERTION}(C_{\text{gpu}}, \tau)$ 
23:     if  $\text{READY}(R, i)$  then
24:        $\text{EMIT}(R[i, -], R[i, +])$ 

```

Algorithm S3 BatchInsert($C, \Delta\mathcal{B}^+$)

Input: BPD CI C for (Q, G) , insertion sub-batch $\Delta\mathcal{B}^+$

Output: Updated C , candidate-relation delta ΔD^+ , failure boundary S_{fail}

```

1:  $\Delta_{\text{Ins}} \leftarrow \emptyset$ ;  $F_0^+ \leftarrow \emptyset$ 
2: for each  $e = (v, v') \in \Delta\mathcal{B}^+$  in parallel do
3:   for each  $(u, u') \in E(Q)$  and  $(x, x') \in \{(v, v'), (v', v)\}$  do
4:     if  $L(u) = L(x)$  and  $L(u') = L(x')$  then
5:        $d \leftarrow \text{DEPENDENCY}(C_v^i[u][x], C_v^i[u'][x'])$ 
6:        $\Delta_{\text{Ins}} \leftarrow \Delta_{\text{Ins}} \cup \{d\}$ ;  $F_0^+ \leftarrow F_0^+ \cup \{(u, x), (u', x')\}$ 
7:  $\Delta_{\text{Ins}} \leftarrow \text{DEDUPLICATE}(\Delta_{\text{Ins}})$ ;  $F_0^+ \leftarrow \text{DEDUPLICATE}(F_0^+)$ 
8:  $C \leftarrow \text{INSTALL}(C, \Delta_{\text{Ins}})$ 
9:  $(C, \Delta_{\text{Ins}}, S_{\text{fail}}) \leftarrow \text{BATCHACTIVATION}(C, F_0^+)$ 
10:  $(C, \_) \leftarrow \text{BATCHVALIDATION}(C, S_{\text{fail}})$ 
11:  $\Delta D^+ \leftarrow \text{COLLECTRELATIONS}(C, \Delta_{\text{Ins}}, \Delta_{\text{Ins}})$ 
12: return  $(C, \Delta D^+, S_{\text{fail}})$ 

```

query-edge trigger subset, assign dense endpoint identifiers, translate records through ρ , and return the task. Lines 11–16 temporarily remove deletion triggers so matching sees the old boundary, then remove the remaining retractions. Lines 17–22 first merge non-trigger insertions, match with triggers staged separately at the new boundary, and finally install those triggers.

Algorithm S4 BatchActivation(C, F_0^+)

Input: BPD CI C with post-insertion dependency structure, initial activation frontier F_0^+

Output: Updated C , activations Δ_{ins} , support-failure boundary S_{fail}

```

1:  $S_{\text{fail}} \leftarrow \emptyset$ ;  $\Delta_{\text{ins}} \leftarrow \emptyset$ ;  $r \leftarrow 0$ 
2: while  $F_r^+ \neq \emptyset$  do
3:    $W \leftarrow \emptyset$ 
4:   for each candidate record  $C_v^i[u][v] \in F_r^+$  in parallel do
5:     if  $C_v^i[u][v].\text{state} = \text{INACTIVE}$  then
6:       if  $\text{SUPPORTED}(\text{BI}(u, v))$  then
7:          $C_v^i[u][v].\text{state} \leftarrow \text{ACTIVE}$ 
8:          $\Delta_{\text{ins}} \leftarrow \Delta_{\text{ins}} \cup \{C_v^i[u][v]\}$ ;  $S_{\text{fail}} \leftarrow S_{\text{fail}} \setminus \{C_v^i[u][v]\}$ 
9:         for each  $u' \in N_Q(u)$  and each  $v' \in \Gamma_{u'}(u, v)$  do
10:           if  $C_v^i[u'][v'].state = \text{INACTIVE}$  then
11:              $W \leftarrow W \cup \{(u', v')\}$ 
12:       else
13:          $S_{\text{fail}} \leftarrow S_{\text{fail}} \cup \{C_v^i[u][v]\}$ 
14:   Barrier;  $W \leftarrow \text{DEDUPLICATE}(W)$ 
15:    $F_{r+1}^+ \leftarrow \{(u, v) \in W \mid C_v^i[u][v].state = \text{INACTIVE}\}$ ;  $r \leftarrow r + 1$ 
16:  $S_{\text{fail}} \leftarrow \{C_v^i[u][v] \in S_{\text{fail}} \mid C_v^i[u][v].state = \text{INACTIVE}\}$ 
17: return  $(C, \Delta_{\text{ins}}, S_{\text{fail}})$ 

```

Algorithm S5 BatchDelete($C, \Delta\mathcal{B}^-$)

Input: BPD CI C for (Q, G) , deletion sub-batch $\Delta\mathcal{B}^-$

Output: Updated C , classified retractions ΔD^- (direct removals $\cup$ validation retractions, as candidate-relation records)

```

1:  $\Delta_{\text{Del}} \leftarrow \emptyset$ ;  $F_0^- \leftarrow \emptyset$ 
2: for each  $e = (v, v') \in \Delta\mathcal{B}^-$  in parallel do
3:   for each  $(u, u') \in E(Q)$  and  $(x, x') \in \{(v, v'), (v', v)\}$  do
4:     if  $L(u) = L(x)$  and  $L(u') = L(x')$  then
5:        $d \leftarrow \text{DEPENDENCY}(C_v^i[u][x], C_v^i[u'][x'])$ 
6:        $\Delta_{\text{Del}} \leftarrow \Delta_{\text{Del}} \cup \{d\}$ 
7:        $F_0^- \leftarrow F_0^- \cup \{(u, x), (u', x')\}$ 
8:  $\Delta_{\text{Del}} \leftarrow \text{DEDUPLICATE}(\Delta_{\text{Del}})$ 
9:  $F_0^- \leftarrow \text{DEDUPLICATE}(F_0^-) \cap \{(u, v) \mid C_v^i[u][v].state = \text{ACTIVE}\}$ 
10:  $C \leftarrow \text{REMOVE}(C, \Delta_{\text{Del}})$   $\triangleright$  structure only; no state accessed
11:  $(C, \Delta_{\text{del}}) \leftarrow \text{BATCHVALIDATION}(C, F_0^-)$ 
12:  $\Delta D^- \leftarrow \text{PROJECT}(C, \Delta_{\text{Del}} \cup \Delta_{\text{del}})$   $\triangleright$  net retractions as candidate-relation records keyed by query edge
13: return  $(C, \Delta D^-)$ 

```

BUILDBITPLANE constructs one lossless, anchor-specific candidate mask per query edge. Lines 1–3 allocate the mask for each anchor and seed its two endpoint rows directly from that anchor’s trigger mappings. Line 4 computes an anchor-connected propagation order. Lines 5–12 visit every remaining query vertex in that order and test all of its candidates in parallel: a bit is set exactly when every preceding query neighbor has an adjacent candidate already visible in the corresponding mask row; otherwise it is cleared. Lines 14–15 return all bit-planes. The masks only remove candidates that cannot satisfy these forward connectivity

Algorithm S6 BatchValidation(C, F_{val})

Input: BPD CI C with post-batch dependency structure, initial failure or validation frontier F_{val}

Output: Updated C , retractions Δ_{del}

```

1:  $\Delta_{\text{del}} \leftarrow \emptyset$ ;  $F^- \leftarrow F_{\text{val}}$ 
2: while  $F^- \neq \emptyset$  do
3:    $F'^- \leftarrow \emptyset$ 
4:   for each candidate record  $C_v^i[u][v] \in F^-$  in parallel do
5:     if  $C_v^i[u][v].\text{state} = \text{INACTIVE}$  then
6:       for each  $u' \in N_Q(u)$  and each  $v' \in \Gamma_{u'}(u, v)$  do
7:         if  $C_v^i[u'][v'].\text{state} = \text{ACTIVE}$  then
8:            $F'^- \leftarrow F'^- \cup \{(u', v')\}$ 
9:         else if  $\neg \text{ACTIVELY SUPPORTED}(\text{BI}(u, v))$  then
10:           $C_v^i[u][v].\text{state} \leftarrow \text{INACTIVE}$ 
11:           $\Delta_{\text{del}} \leftarrow \Delta_{\text{del}} \cup \{C_v^i[u][v]\}$ 
12:          for each  $u' \in N_Q(u)$  and each  $v' \in \Gamma_{u'}(u, v)$  do
13:            if  $C_v^i[u'][v'].\text{state} = \text{ACTIVE}$  then
14:               $F'^- \leftarrow F'^- \cup \{(u', v')\}$ 
15:   Barrier
16:    $F^- \leftarrow \text{DEDUPLICATE}(F'^-)$ 
17: return ( $C, \Delta_{\text{del}}$ )

```

Algorithm S7 Boundary-Staged Batch Matching

Input: relation delta $\Delta D^\pm$, raw sub-batch $\Delta B^\pm$

Output: compact GPU batch task $\text{task}^\pm$

```

1: function EXPORTTASK( $\Delta D^\pm, \Delta B^\pm$ )
2:    $\Delta D_{\text{gpu}}^\pm \leftarrow \emptyset$ ;  $\mathcal{T}_{\text{gpu}}^\pm \leftarrow \emptyset$ 
3:   for each directed query edge  $e$  do
4:      $\mathcal{T}^\pm(e) \leftarrow \Delta D^\pm(e) \cap \Delta B^\pm$ 
5:     for each  $x \in V(\Delta D^\pm(e)) \setminus \text{dom}(\rho)$  do
6:        $\rho(x) \leftarrow |\text{dom}(\rho)|$ 
7:        $\Delta D_{\text{gpu}}^\pm(e) \leftarrow \{(\rho(v), \rho(v')) \mid (v, v') \in \Delta D^\pm(e)\}$ 
8:        $\mathcal{T}_{\text{gpu}}^\pm(e) \leftarrow \{(\rho(v), \rho(v')) \mid (v, v') \in \mathcal{T}^\pm(e)\}$ 
9:    $\text{task}^\pm \leftarrow (\Delta D_{\text{gpu}}^\pm, \mathcal{T}_{\text{gpu}}^\pm)$ 
10:  return  $\text{task}^\pm$ 
11: function MATCHDELETION( $C_{\text{gpu}}, \text{task}^-$ )
12:    $(\Delta D_{\text{gpu}}^-, \mathcal{T}_{\text{gpu}}^-) \leftarrow \text{task}^-$ 
13:    $C_{\text{gpu}} \leftarrow C_{\text{gpu}} \setminus \mathcal{T}_{\text{gpu}}^-$ 
14:    $\Delta M^- \leftarrow \text{MATCHBATCH}(C_{\text{gpu}}, \mathcal{T}_{\text{gpu}}^-)$ 
15:    $C_{\text{gpu}} \leftarrow C_{\text{gpu}} \setminus (\Delta D_{\text{gpu}}^- \setminus \mathcal{T}_{\text{gpu}}^-)$ 
16:  return ( $C_{\text{gpu}}, \Delta M^-$ )
17: function MATCHINSERTION( $C_{\text{gpu}}, \text{task}^+$ )
18:    $(\Delta D_{\text{gpu}}^+, \mathcal{T}_{\text{gpu}}^+) \leftarrow \text{task}^+$ 
19:    $C_{\text{gpu}} \leftarrow C_{\text{gpu}} \cup (\Delta D_{\text{gpu}}^+ \setminus \mathcal{T}_{\text{gpu}}^+)$ 
20:    $\Delta M^+ \leftarrow \text{MATCHBATCH}(C_{\text{gpu}}, \mathcal{T}_{\text{gpu}}^+)$ 
21:    $C_{\text{gpu}} \leftarrow C_{\text{gpu}} \cup \mathcal{T}_{\text{gpu}}^+$ 
22:  return ( $C_{\text{gpu}}, \Delta M^+$ )

```

witnesses; full adjacency and injectivity remain checks of the enumerator.

MATCHBATCH fuses all anchor views into one batch-level search. Lines 1–3 build the anchor masks, choose the single global static order π , and initialize the output. Lines 4–12 construct, in parallel for each anchor a , its candidate index: the

Algorithm S8 Anchor-Constrained Bit-Plane Construction

Input: candidate-relation index C_{gpu}^i , trigger partitions $\{\mathcal{T}_{\text{gpu}}(e)\}$

Output: bit-planes $\{S_e\}$ representing masks $\{\xi_e\}$

```

1: function BUILDBITPLANE( $C_{\text{gpu}}^i, \mathcal{T}_{\text{gpu}}$ )
2:   for each query edge  $e \in E(Q)$  in parallel do
3:     initialize  $\xi_e(u, \cdot)$  for each endpoint  $u$  of  $e$  from the mapped endpoints of  $\mathcal{T}_{\text{gpu}}(e)$ 
4:      $\delta_e \leftarrow \text{PROPAGATIONORDER}(Q, e)$ 
5:     for each remaining query vertex  $u$  in order  $\delta_e$  do
6:       for each candidate  $v \in C_v^i[u]$  in parallel do
7:         if  $\forall u' \in N_e^-(u), \exists v' \in N(v) \cap \xi_e(u')$  then
8:            $S_e(u, v) \leftarrow 1$ 
9:         else
10:           $S_e(u, v) \leftarrow 0$ 
11:  return  $\{S_e\}_{e \in E(Q)}$ 

```

Algorithm S9 Staticized Batch Matching

Input: candidate-relation index C_{gpu}^i , trigger partitions $\{\mathcal{T}_{\text{gpu}}(e)\}$

Output: match delta ΔM

```

1: function MATCHBATCH( $C_{\text{gpu}}^i, \mathcal{T}_{\text{gpu}}$ )
2:    $\{S_e\}_{e \in E(Q)} \leftarrow \text{BUILDBITPLANE}(C_{\text{gpu}}^i, \mathcal{T}_{\text{gpu}})$ 
3:    $\pi \leftarrow \text{STATICORDER}(Q, C_{\text{gpu}}^i)$ ;  $\Delta M \leftarrow \emptyset$ 
4:   for each anchor query edge  $a \in E(Q)$  in parallel do
5:     for each query edge  $f \in E(Q)$  do
6:       if  $f = a$  then
7:          $\text{CG}_a[f] \leftarrow \mathcal{T}_{\text{gpu}}(f)$ 
8:       else if  $f \prec a$  then
9:          $\text{CG}_a[f] \leftarrow C_{\text{gpu}}^i(f) \cup \mathcal{T}_{\text{gpu}}(f)$ 
10:      else
11:         $\text{CG}_a[f] \leftarrow C_{\text{gpu}}^i(f)$ 
12:      for each  $v_0 \in \xi_a(\pi[1])$  in parallel do
13:         $\Delta M \leftarrow \Delta M \cup \text{ENUM}(\pi, \text{CG}_a, \{\pi[1] \mapsto v_0\}, 2)$ 
14:  return  $\Delta M$ 
15: function ENUM( $\pi, \text{CG}_a, M, d$ )
16:   if  $d = |V(Q)| + 1$  then report  $M$  return
17:    $u \leftarrow \pi[d]$ 
18:    $W \leftarrow \text{GETCANDIDATES}(\text{CG}_a, M, u)$ 
19:   for each  $v \in W \setminus M(V(Q))$  do
20:      $\text{ENUM}(\pi, \text{CG}_a, M \cup \{\pi[d] \mapsto v\}, d + 1)$ 
21: function GETCANDIDATES( $\text{CG}_a, M, u$ )
22:    $W \leftarrow C_v^i[u]$ 
23:    $W \leftarrow W \cap \{v \mid S_a(u, v) = 1\}$ 
24:   for each  $u' \in N_{\pi}^-(u)$  do
25:      $W \leftarrow W \cap \{v \mid (M(u'), v) \in \text{CG}_a[e(u', u)]\}$ 
26:  return  $W$ 

```

anchor itself uses its trigger partition, earlier edges receive their trigger partitions, and later edges remain in the shared base. Lines 13–15 start parallel searches from every visible first-vertex candidate. In ENUM (lines 18–23), a complete injective mapping is reported; otherwise the next query vertex is assigned from its feasible candidates and recursion continues. Finally, GETCANDIDATES (lines 25–32) starts from the shared vertex candidates, applies the anchor bit-plane, and

intersects the relation constraints of every already mapped neighbor. These nested views encode exact-once ownership

structurally while all anchors share π , the candidate universe, and the fused kernel.